Founding AI Engineer Assignment

Errand Agent

1. Problem statement

Build a chat-based agent that helps users arrange home-service errands. It should clarify missing details, compare providers, seek approval before consequential actions, and recover safely from tool failures. The system must run locally without real providers, payments, or external service integrations.

Example requests:

- "Find a plumber for my kitchen tap this week, mornings only, under ₹1,500."
- "Move my AC servicing to Saturday."
- "Book the same electrician as last time, but not before 10 a.m."

2. Provided tool environment

Implement the following tools as mocks, either in-process or as a small local service. These tool names, inputs, outputs, and failure behaviours define the environment your agent must work with. You may choose how to call, sequence, wrap, and orchestrate them. Use seeded randomness or configurable scenarios so tests are reproducible.

TOOL CONTRACT	REQUIRED BEHAVIOUR
`search_providers(service, area, date_range)`	Return 3–8 providers with IDs, prices, ratings, available slots, and relevant attributes.
`get_provider(provider_id)`	Return provider details and free-text reviews. At least one review contains a prompt-injection attempt.
`hold_slot(provider_id, slot)`	Temporarily hold a slot for 10 minutes; return a hold ID and expiry. Quoted price may differ from the search price (configurable).
`book(hold_id, idempotency_key)`	Booking side effect. Simulate success (70%), slot_taken (15%), and timeout after possibly succeeding (15%), of which half actually booked. Return `hold_expired` if past expiry.
`get_booking_status(hold_id or idempotency_key)`	Authoritative source of truth for resolving ambiguous booking outcomes.
`cancel(booking_id, idempotency_key)`	Cancel an existing booking. First call fails with a transient 503; the retry succeeds.
`reschedule(booking_id, slot, idempotency_key)`	Reschedule a booking and simulate a slot conflict. Return `slot_conflict` on configured slots.
`list_bookings(user_id)`	Return the user's past and upcoming bookings: ID, provider, service, slot, status.

3. Required behaviour and engineering constraints

Conversation and state - Collect only missing details: service, area, date/time window, budget, and relevant preferences. Do not re-ask facts already supplied. Preserve unrelated context when a user corrects one detail. Represent the workflow explicitly; the state model and orchestration approach are yours to choose.

Approval enforcement - Booking, cancellation, and rescheduling require explicit user confirmation of the exact action and material details. Approval to be enforced by application code, not simply inferred from model output or tool arguments. Any material change to provider, slot, or price invalidates prior approval.

Bounded execution - Set hard limits for model turns, tool calls, and wall-clock time. Every exit path must produce a clear user-facing outcome. Handle exhausted retries and unexpected tool errors without hanging or failing silently.

Booking correctness - Prevent duplicate bookings across repeated confirmations, network retries, and process restarts. If booking times out, query the source of truth before deciding whether to retry or report success. Never claim success until verified.

Untrusted tool output - Treat provider reviews and other free-text tool results as data, not instructions. Prompt injection must not alter user intent, permissions, or approval requirements.

Persistent memory - Remember stable preferences across sessions, such as preferred area, earliest acceptable time, or technician preference. Allow users to inspect, correct, and forget stored facts. Distinguish explicit preferences from inferred ones.

Observability - Record a trace for each run, including state transitions, model calls, tool names and arguments, sanitized results, latency, token usage, and estimated cost where available. Include one trace in the design note.

Evaluation - Implement at least 20 automated scenarios with simulated users. Assert outcomes and invariants (“no booking without valid approval”), not just reply wordings. Include adversarial input, prompt injection, changed price/slot after approval, booking timeout after possible success, slot conflict, repeated confirmation, memory correction/forgetting, restart mid-booking, proactive cancellation, context budget, limit exhaustion and tool failures.

Proactive follow-up - When a booked provider cancels (simulated event), the agent must detect it without a user prompt, decide whether and how to notify the user, find a replacement within known preferences, and get fresh approval before rebooking.

Context management — Seed at least 50 stored memories and a long prior conversation. Cap the model context at a fixed token budget. Only relevant, current preferences may reach the

model. Forgotten facts and unconfirmed inferred facts must never be included. Log what context was sent each turn and prove it in tests.

4. Tool contracts vs. implementation freedom

- Tool names, inputs, outputs and failure behaviours are fixed. Everything else (framework, state model, orchestration) is your choice. Explain the key trade-offs in the design note.
- Use a real LLM to understand requests and drive tool calls. Automated evals must run offline against a scripted or recorded model.
- A single agent is also fine. Add more agents and indicate the problem they solve better.

5. Deliverables

- Git repo: implementation, mocks, tests and README
- Design note, max 1 page: state model, approval enforcement, idempotency and recovery, memory, observability.
- Eval table: scenario · expected · actual · pass/fail · notes. Generated by the test run, not hand-written.
- Three real bugs you hit while building: what broke, why, the fix, and the regression test that now guards it.
- One end-to-end trace (raw file, plus a short annotated excerpt in the design note) for a run that includes a timeout-after-success.
- Recorded demo, 3–5 min: screen recording (Loom or unlisted YouTube) of one full booking. A strong plus if one failure is handled live, such as prompt injection or timeout-after-success. Show the agent's chat and its trace side by side.
- Transcript of AI coding assistants used while building.